CSC 4553

Fall 2026 - Day 22

Admin

- Assignment 2 due tonight
- Quiz / Lab, as usual

Directories

Questions to consider

- Why do hierarchical directories solve problems that flat directories cannot?
- What is the difference between a hard link and a soft link?
- What happens when you delete a file that has multiple hard links? A soft link?

Directories

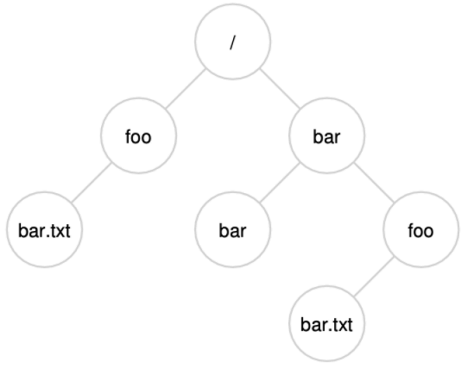
- Why don't we use a single (flat) directory for all users?
  - → Naming problem: each file name must be unique
  - → Grouping problem: how to remember what files are mine / relate to my current coding project?
- What if we just had a single directory for each user but stayed flat otherwise?
  - → You still have a naming problem. How many `Makefiles` do you have?

What is a directory?

- **Directories** are a special kind of file, whose data are `<name, inode>` pairs, called **directory entries** (a.k.a. dirents)
- Only direct children: a path like `/a/b/c` is resolved one component at a time
- By default, a directory contains `.` (self reference) and `..` (parent)

Directory structure

- How do we want to structure directories?
- Common approach: a hierarchy (tree)
  - → UNIX: a tree starting with root (“/”)
- Paths: absolute or relative. Examples?
  - → `/home/prs/file.txt` vs. `./file.txt`



Directory operations

- What operations do we want to support on directories?
- Common operations:
  - → `mkdir`, `rmdir`, `opendir`, `closedir`, `readdir`, `rename`, `link`, `unlink`
- `rename`: remove the old directory entry and add a new one (atomically); the file's data doesn't move
  - → Parent directories' modification times change
- `link`: add a new directory entry pointing to the *same* inode
  - → The file's data is untouched, but the inode's link count goes up (and its change time is updated)
- `unlink`: remove a directory entry and decrement the link count

Hard and soft links

- Who can tell me the difference?
  - → **Hard**: another directory entry for the same inode
  - → **Soft**: a separate file whose data is a path
- How do you create them in Linux?
  - → `ln target name` vs. `ln -s target name`
- Hard links can't span file systems or point to directories (no cycles); symlinks can, so they can loop

Predict the links

```
echo hello > target
ln target hard
ln -s target soft
ls -li          # (1)
rm target
cat hard        # (2)
cat soft        # (3)
echo bye > target
cat hard        # (4)
cat soft        # (5)
```

- **Think**: At (1), which names share an inode, and what is each link count? Predict the output of (2) and (3).
- **Discuss**: Compare with a neighbor, then predict (4) and (5). Why do `hard` and `soft` now disagree?

What isn’t clear?

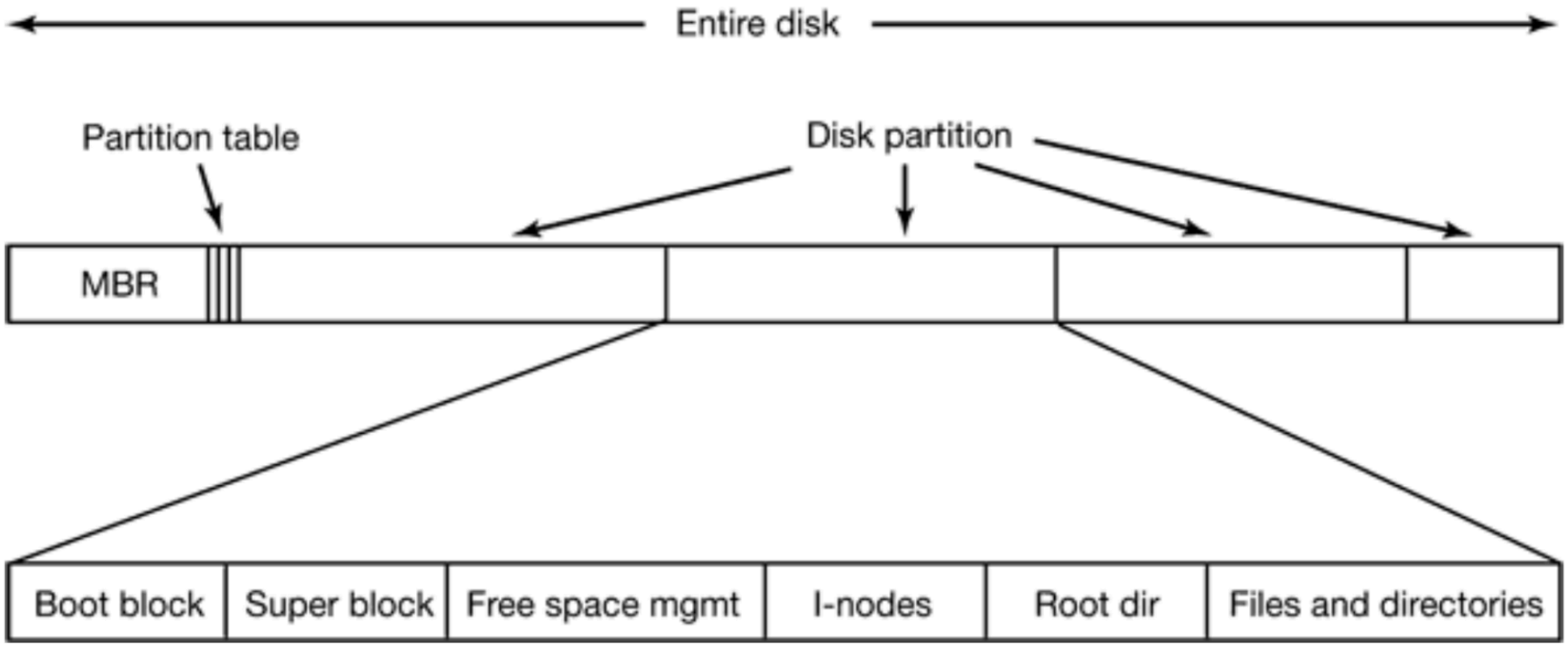
Comments? Thoughts?

File system structures

Questions to consider

- How is a disk partitioned and bootstrapped so the OS can find its file systems?
- What on-disk structures are needed to organize files, and what role does each play?

Disk layout

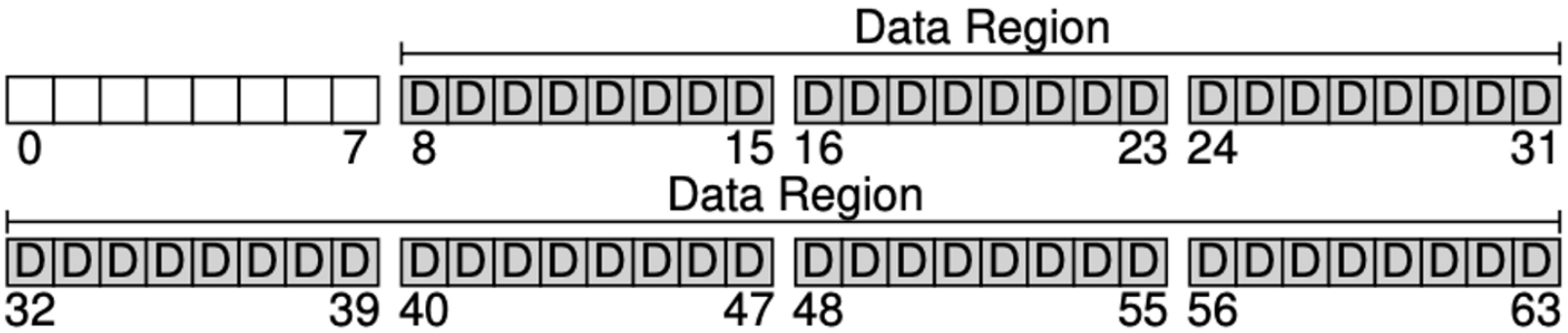

Entire disk
Partition table
Disk partition
MBR
Boot block
Super block
Free space mgmt
I-nodes
Root dir
Files and directories

Disk layout (cont'd)

- **Master Boot Record** (MBR): sector 0 holds the partition table and boot code
  - → BIOS reads MBR, locates the active partition, loads its boot block, and jumps to the OS loader
  - → Modern machines: GPT partition table + UEFI firmware, same idea
- File system structures vary by implementation; two major on-disk components:
  - → **Metadata**: information about files (inodes, bitmaps, superblock)
  - → **Data**: the actual file contents

Start with a blank disk

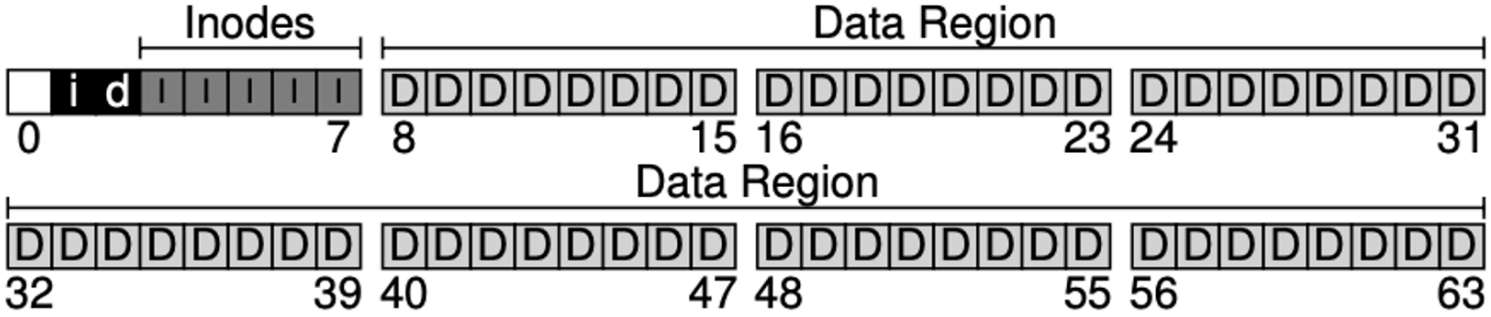
- Divided into blocks, let's say 64 blocks that are 4KB each
- Most of the disk is reserved for **data blocks** (storing file contents)



Metadata blocks

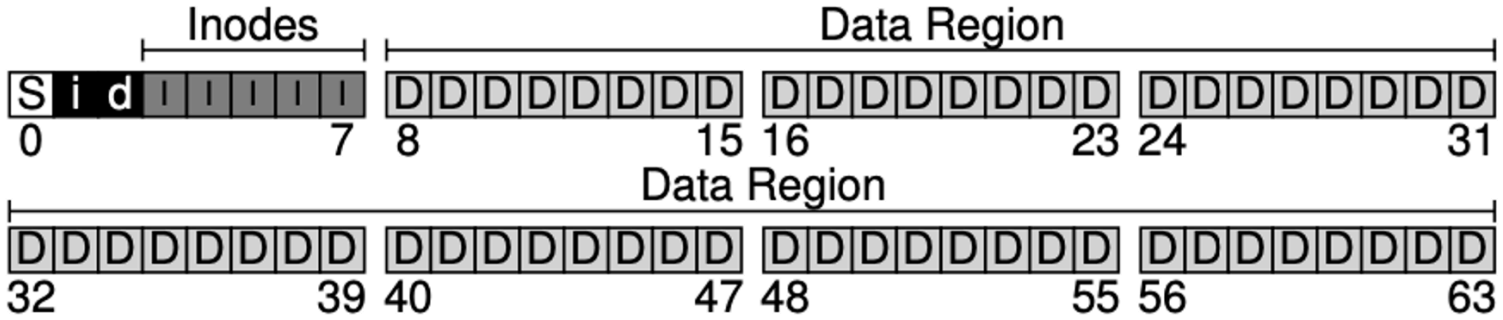
- Metadata about files is stored in **inodes**, grouped into an inode table
- We also need to track which blocks are in use: **bitmaps** (0 = free, 1 = in-use)

  → One bitmap block for inodes, one for data blocks



How do we bootstrap the file system?

- When the system starts, how do we figure out how to do things with this file system?
  - → Superblock



Superblock

- The superblock describes the file system as a whole:
  - → Type (a magic number identifying e.g. ext4)
  - → Block and inode counts (total and free)
  - → Locations of the bitmaps, inode table, and data blocks
  - → Mount state (clean or dirty)
- Without it the file system can’t be mounted, so it’s replicated in several places on disk

What isn’t clear?

Comments? Thoughts?

Inodes

Questions to consider

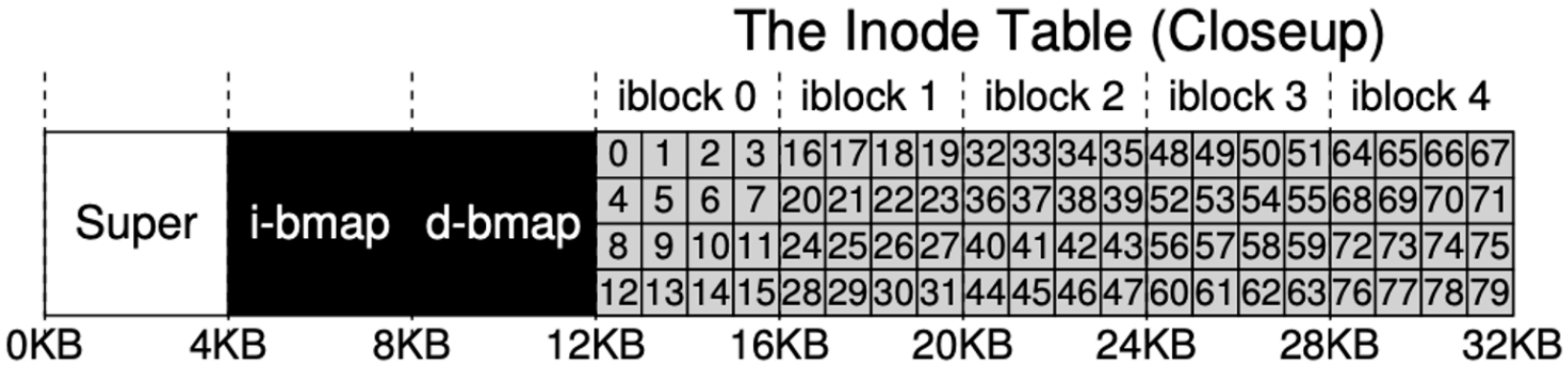
- What information does an inode store, and why doesn't it include the file name?
- Why can't we just use direct pointers for all files, and how do we solve the problem?

Inodes

- Each inode tracks metadata about a file, including:
  - → Pointers to data blocks associated with the file
  - → **Link count** (number of directory entries that point to this inode)
  - → Permissions, owner and group IDs, file size
  - → Timestamps (change, modification, access)
  - → (most things available from `stat()`)
- Does **NOT** contain the file name
  - → Names live in directory entries, so one inode can have many names (hard links)

Inodes (cont'd)

- Users see names; the OS works with inode numbers
- Let's imagine inodes are 256 bytes, so we can store 80 in the example 20KB table



Inodes (cont'd)

- We use the inode to reference the file's data blocks, which are separate from the inode itself
- How should we track the data blocks for a file?
  - → One option: direct pointers
    - Any problems with this approach?

Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
2	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
4	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists

Figure 40.1: **Simplified Ext2 Inode**

Multi-level pointers

- Direct pointers alone limit file size. The ext2 fix: 15 pointers in the inode
  - → 12 **direct**: point to data blocks
  - → 1 **single indirect**: a block of pointers to data blocks
  - → 1 **double indirect**: a block of pointers to single indirect blocks
  - → 1 **triple indirect**: one more level
- 4KB blocks, 32-bit (4-byte) block addresses

- **Think**: What is the largest file reachable using only direct pointers? Up through single, double, and triple indirect?
- **Discuss**: The inode is already in memory. How many disk reads to read the byte at offset 10,000? At 5,000,000? At 5GB?

Multi-level pointers: answers

- Direct only: $12 * 4KB = 48KB$
- Single indirect: $(12 + 1024) * 4KB = 4144KB$
- Double indirect: $(12 + 1024 + 1024^2) * 4KB =\sim 4GB$
- Triple indirect: $(12 + 1024 + 1024^2 + 1024^3) * 4KB =\sim 4TB$
- Reads: offset 10,000 needs 1, offset 5,000,000 needs 3, offset 5GB needs 4

Inodes (cont'd)

- Multi-level pointers support a wide range of file sizes with a fixed-size inode
  - → Used by ext2, ext3, and others
- Downside: big files need up to 4 reads per data block (as you just computed). So why is this used?
  - → The vast majority of files are *small*, so optimizing for that case is fine

What isn’t clear?

Comments? Thoughts?

File system component implementations

Questions to consider

- What data structures can be used to implement directories, and what are the trade-offs of each?
- What are the trade-offs between contiguous, linked, and indexed data block allocation?
- What are the options for tracking free blocks, and when does each make sense?

Directory implementation

- A directory is a file of `<name, inode>` entries; we need to search, add, delete, and rename
- Simplest structure: a linear list
  - → Easy to implement
  - → Downsides?
    - Lookups are O(n), and we hit directories on every path lookup. Any way to speed it up?
      - Caching: many OSes cache recent directory info
      - Sorted lists / balanced trees: faster search, but keeping them sorted and balanced adds complexity

Directory implementation (cont'd)

- Hash table: hash the file name to find its entry
  - → Pro: no more linear searching
  - → Any complications?
    - Collisions are inevitable, so we need chaining or probing
    - Fixed size: past the limit, the table has to grow
  - → Why not just use a giant hash table by default?
    - Small tables fit in hardware caches, and most directories are small

Data block allocation

- Where do a file's blocks go? No single best answer

Contiguous

- Pro: fast sequential, simple
- Con: external fragmentation, hard to grow
- *Extents*: runs of blocks

Linked

- Pro: no external fragmentation, easy to grow
- Con: slow random access; one bad pointer loses the rest

Indexed

- Index block of pointers to data blocks
- The inode scheme from earlier

Free block tracking

- **Bitmaps**: one bit per block (0 = free, 1 = in-use)
  - → Space-efficient (32MB for a 1TB drive at 4KB blocks), fast bitwise scanning
  - → Used by ext2/3/4, NTFS
  - → Con: finding *contiguous* free blocks requires scanning
- Modern large-scale file systems use **extent trees** (XFS, Btrfs) or **B-trees** (ZFS) that track *runs* of free blocks rather than individual ones

What isn’t clear?

Comments? Thoughts?